Методические указания к лабораторной работе

Автор: доцент НИТПУ Друки А.А.

Работа с данными с помощью Apache Spark

Данная лабораторная работа посвящена работе с табличными данными с помощью фреймворка Apache Spark на платформе Google Colab.

Цель работы: получить навыки работы с табличными данными с помощью фреймворка Apache Spark в виде SQL запросов и используя модели машинного обучения.

Задачи:

1. Получение навыков работы с табличными данными с помощью Apache Spark SQL в виде SQL запросов;
2. Получение навыков работы с табличными данными с помощью Apache Spark MLlib используя модели машинного обучения.

Apache Spark

Apache Spark – это распределенный фреймворк для обработки данных, который предоставляет высокую производительность и удобные инструменты для анализа больших объемов данных. Он поддерживает различные типы задач обработки данных, включая обработку потоков данных, машинное обучение и аналитику данных. Apache Spark обеспечивает высокую скорость обработки данных за счет распределенной обработки в памяти и возможности работы с данными в пакетном и потоковом режимах.

Преимущества Spark:

Ленивые вычисления. Spark использует концепцию отложенного выполнения вычислений. Это означает, что операции над данными проводятся только перед непосредственным использованием результатов этих операций. Благодаря этому вычислительные мощности не тратятся на вычисления, которые понадобятся когда-то в будущем.

Параллельная обработка и комбинирование операций. Spark распределяет данные и вычисления по нескольким узлам в кластере, выполняя разные операции обработки параллельно в режиме реального времени. Это отличает его от MapReduce, при использовании которого каждый следующий этап работы с датасетом требует завершения предыдущего. Благодаря этим особенностям Spark имеет в десятки раз большую скорость работы с данными.

Google Colab

Google Colab (Google Colaboratory) – это бесплатный облачный сервис на основе Jupyter Notebook. Google Colab предоставляет необходимые инструменты для работы с искусственными нейронными сетями прямо в браузере. С помощью данного сервиса можно создавать, обучать и тестировать искусственные нейронные сети для решения различного рода задач. Поддерживает язык программирования Python.

При работе с сервисом:

- не требуется никакой настройки;

- вы получаете бесплатный доступ к процессорам (CPU, GPU и TPU);
- вашей программой можно поделиться с другими пользователями.

Это удобное решение для студентов, специалистов по обработке данных и исследователей в области искусственного интеллекта. Чтобы начать работу с инструментом пройдите по ссылке: https://colab.research.google.com/notebooks/intro.ipynb

Ход работы

1. Создание блокнота в Google Colab

Чтобы начать работать с Google Colab необходимо создать блокнот. Для этого зайдите в Google Colab по ссылке: https://colab.research.google.com/. После этого Вы увидите окно, представленное ниже (рисунок 1).

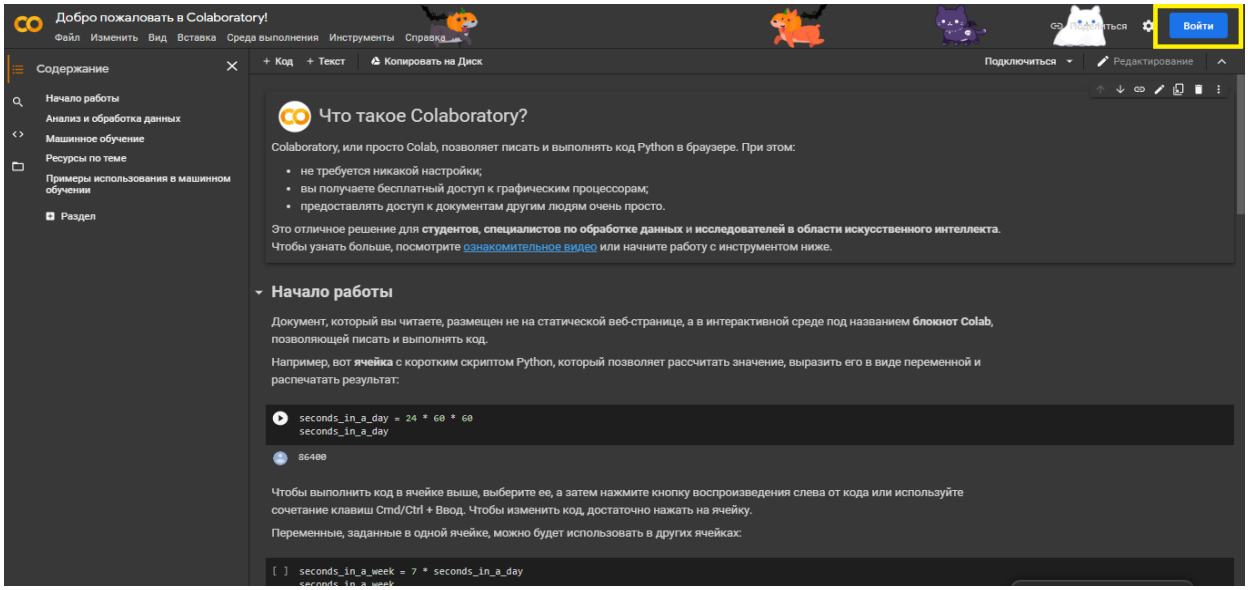


Рисунок 1. Главная страница Google Colab

Для более удобного отображения в методических материалах представлено тёмное оформление пользовательского интерфейса Google Colab. В вашем случае оформление может быть белым. Сменить цвет оформления можно в настройках.

Нажмите кнопку «Войти» в правом верхнем углу (выделено цветом) (рисунок 1).

Далее Вам необходимо пройти процедуру авторизации/регистрации. Если у вас есть аккаунт YouTube или Gmail, то можете зайти под этим аккаунтом.

Для создания новой страницы для написания кода нажмите «Файл» -> «Создать блокнот» (рисунок 2).

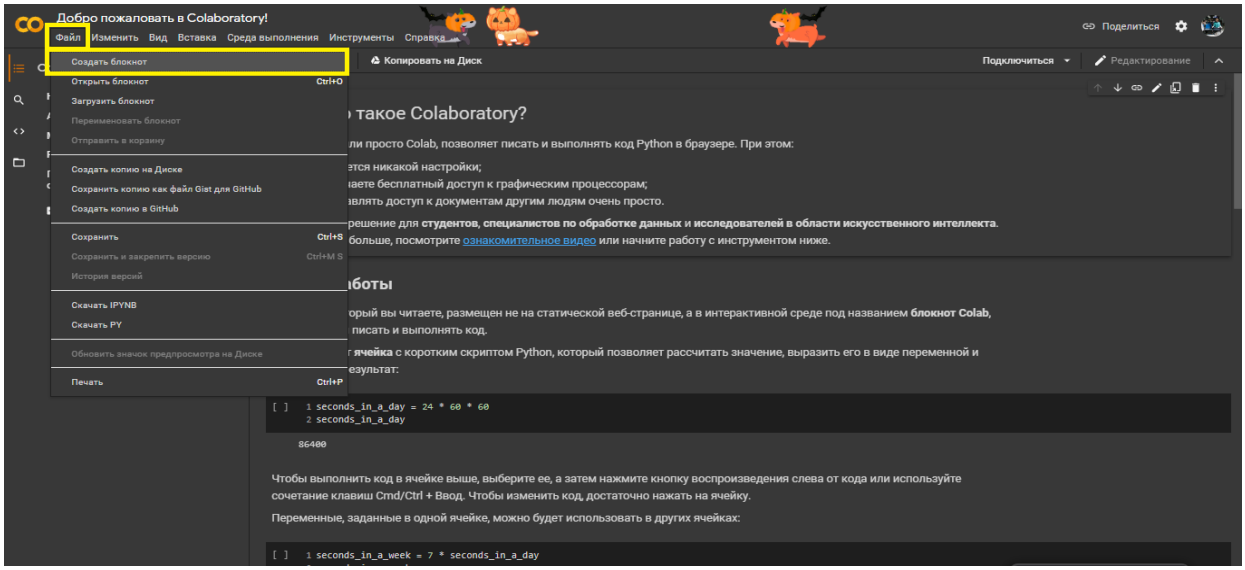


Рисунок 2. Вкладка «Создать блокнот» в Google Colab

Вы создали блокнот Google Colab. Теперь можете приступать к созданию кода (рисунок 3).

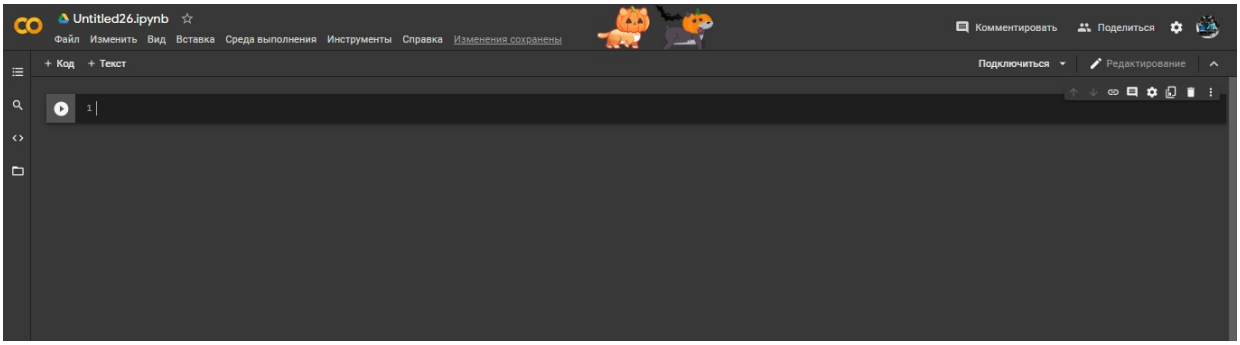


Рисунок 3. Блокнот в Google Colab

Также вам понадобится создать отдельные блоки для того чтобы разделять код по частям. Делается это следующим способом: наведите курсор на нижнюю границу блока кода (или блока с выводом результата кода), появится «+ Код», «+ Текст». Нажмите «+ Код» (или сочетание клавиш «Ctrl+M+B») и добавится ещё один блок.

2. Работа с Apache Spark MLlib

Apache Spark MLlib – это модуль Apache Spark, который включает различные алгоритмы машинного обучения и инструменты для работы с данными, позволяя разработчикам создавать и обучать модели машинного обучения на больших объемах данных. Данный модуль содержит функции для решения задач классификации, регрессии, кластеризации и т.д.

Основные функции Apache Spark MLlib включают:

- Алгоритмы машинного обучения: линейная регрессия, логистическая регрессия, деревья решений, случайные леса, градиентный бустинг и многое другое.
- Инструменты предобработки данных, включая векторизацию, нормализацию, фильтрацию, преобразование признаков и др.
- Поддержка распределенного обучения моделей и предсказаний на больших объемах данных.

Пример работы с Apache Spark MLlib

В качестве набора данных возьмем базу данных здоровья внутриутробного развития плода на основе данных КТГ: https://www.kaggle.com/datasets/andrewmvd/fetal-health-classification Метка класса: fetal_health. Класс отрицательный – 1 (Normal – норма), класс положительный – 2, 3 (Suspect – подозрение, Pathological - патология).

Построим три модели машинного обучения: логистическая регрессия, дерево решений и случайный лес для набора данных. Для каждого алгоритма необходимо получить лучшую модель, путем поиска наилучшего набора её параметров. Для алгоритма логистической регрессии это параметры maxIter=10…10000, regParam>0 (0.1, 0.5, 1, …), elasticNetParam=0…1. Для дерева решений maxDepth > 0 (3, 5, 9, 12, …). Для случайного леса maxDepth > 0 (3, 5, 9, 12, …) и numTree > 0 (5, 11, 25, …). Получить значения матриц ошибок (Confusion Matrix), верности (Accuracy), полноты (Recall) и точности (Precision) для всех моделей.

Для начала работы импортируем необходимые библиотеки и модули.

```
!pip install pyspark
import pyspark
from pyspark.sql import SparkSession
```

Создадим объект SparkSession с именем приложения "Fetal Health Classification".

```
spark = SparkSession.builder.appName("Fetal Health
Classification").getOrCreate()
```

Данный блок кода выполняет чтение файла в формате CSV с пометкой заголовков и выводит содержимое с помощью метода show().

```
df = spark.read.csv('fetal_health.csv', header=True, inferSchema=True)
df.printSchema()
df.show()
```

Выполним предварительную обработку данных. Создается новый столбец с именем "label" в датафрейме df. Этот столбец используется для хранения меток классов. Функция **when** используется для условного присвоения значений в столбце "label": если значение в столбце "fetal_health" равно 1, то значение в столбце "label" будет 0, иначе 1.

```
from pyspark.sql.functions import when

df = df.withColumn("label", when(df.fetal_health == 1, 0).otherwise(1))
```

Удаляет столбец "fetal_health" из датафрейма df, так как он больше не нужен после создания столбца "label". Это делается с помощью метода **drop**, который принимает имя столбца, который нужно удалить.

```
df = df.drop('fetal_health')
```

Выполним разделение данных на обучающую и тестовую выборки с помощью метода **randomSplit**.

```
train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)
```

Аргументы метода **randomSplit**:

- [0.8, 0.2]: это список долей, которые определяют, какую часть данных нужно выделить для обучения (80%) и тестирования (20%).

- **seed** = 42: Этот параметр определяет начальное состояние для генератора случайных чисел. Установка **seed** гарантирует, что при каждом запуске программы будут получены одни и те же результаты разделения данных.

Выполним подготовку признаковых столбцов для обучения моделей машинного обучения. Создадим список **feature_columns**, который содержит имена всех столбцов датафрейма df, за исключением столбца "label". Проходимся по всем именам столбцов df, и если имя столбца не равно "label", оно добавляется в список **feature_columns**.

```
feature_columns = [c for c in df.columns if c != 'label']
```

Создадим объект **VectorAssembler**, который объединяет значения из выбранных признаковых столбцов в один вектор-признак.

inputCols принимает список имен признаковых столбцов, которые мы хотим объединить, в данном случае это список **feature_columns**.

outputCol определяет имя столбца, в который будет записан результирующий вектор-признак. В данном случае этот столбец будет иметь имя "features". Таким образом, после выполнения этого кода объект assembler будет готов к использованию для преобразования выбранных признаковых столбцов в один столбец с вектором-признаком.

```
assembler = VectorAssembler(inputCols=feature_columns, outputCol="features")
```

Для создания моделей машинного обучения импортируем необходимые библиотеки и модули.

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression,
DecisionTreeClassifier, RandomForestClassifier
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
```

Создадим три модели классификации данных: логистическая регрессия, дерево решений, случайный лес.

```
.
.
Допишите код
.
.
```

Зададим параметры моделей с помощью объекта ParamGridBuilder.

```
paramGrid_lr = ParamGridBuilder() \
    .addGrid(lr.maxIter, [10, 100, 1000, 10000]) \
    .addGrid(lr.regParam, [0.1, 0.5, 1.0]) \
    .addGrid(lr.elasticNetParam, [0.0, 0.5, 1.0]) \
    .build()
```

maxIter – максимальное количество итераций алгоритма оптимизации.

regParam – параметр регуляризации.

elasticNetParam – параметр смешивания L1 и L2 регуляризации.

```
paramGrid_dt = ParamGridBuilder() \
    .addGrid(dt.maxDepth, [3, 5, 9, 12]) \
    .build()
```

maxDepth – максимальная глубина дерева.

```
paramGrid_rf = ParamGridBuilder() \
    .addGrid(rf.maxDepth, [3, 5, 9, 12]) \
    .addGrid(rf.numTrees, [5, 11, 25]) \
    .build()
```

maxDepth – максимальная глубина деревьев в лесу.

numTrees – количество деревьев в лесу.

Каждый вызов метода **addGrid** добавляет к сетке параметров новую комбинацию значений для заданного гиперпараметра. После добавления всех комбинаций значений с помощью метода **build()** создается окончательная сетка параметров, которая будет использоваться в процессе поиска наилучших гиперпараметров модели.

Создадим три кросс-валидатора (**cv_lr**, **cv_dt**, **cv_rf**), которые будут использоваться для обучения моделей. Каждый кросс-валидатор принимает на вход **estimator** моделей (lr, dt, rf), набор параметров модели **paramGrid**, **evaluator** (**MulticlassClassificationEvaluator** для оценки качества классификации) и число фолдов **numFolds** для кросс-валидации. **labelCol="label"** указывает столбец меток в данных.

```
.
.
Допишите код
.
.
```

Создадим три конвейера (pipelines) для обучения моделей. Сначала данные проходят через этап сборки признаков (**assembler**), а затем проходят через процесс кросс-валидации

```
.
.
Допишите код
.
.
```

Выполним обучение моделей.

```
.
.
Допишите код
.
.
```

Создадим функцию для оценки моделей на тестовых данных.

```
.
```

```
.
Допишите код
.
.
```

Оценка метрик классификации. Здесь используются метки из столбца "label" в тестовых данных и прогнозы модели из столбца "prediction".

```
.
.
Допишите код
.
.
```

Оценка точности модели на основе прогнозов и истинных меток.

```
.
.
Допишите код
.
.
```

Функция возвращает четыре значения: точность (accuracy), взвешенную точность (weighted precision), взвешенную полноту (weighted recall) и F1-меру, позволяя оценить производительность модели по нескольким метрикам.

Выполним вызов функции и сохраним результаты оценки в четырех соответствующих переменных:

```
.
.
Допишите код
.
.
```

Выведем на экран полученные результаты.

```
print("Логистическая регрессия - Accuracy = {}, Precision = {}, Recall = {}, F1 = {}".format(accuracy_lr, precision_lr, recall_lr, f1_lr))
print("Дерево решений - Accuracy = {}, Precision = {}, Recall = {}, F1 = {}".format(accuracy_dt, precision_dt, recall_dt, f1_dt))
print("Случайный лес - Accuracy = {}, Precision = {}, Recall = {}, F1 = {}".format(accuracy_rf, precision_rf, recall_rf, f1_rf))
```

Задание

В качестве заданий для самостоятельной работы выберите один вариант из представленных ниже:

1. Оценка вероятности диагностики диабета у человека: https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset Метка класса: Diabetes_012. Класс отрицательный – 0 (no diabetes – нет диабета), класс положительный – 1; 2 (prediabetes – преддиабетическое состояние; diabetes - диабет).

2. Оценка вероятности, того, что клиент откроет банковский депозит в результате маркетинговой акции: https://archive.ics.uci.edu/ml/datasets/Bank+Marketing Метка класса: y. Класс отрицательный – 'yes', Класс положительный – 'no'.

3. Оценка уровня физического развития людей разного возраста: https://www.kaggle.com/datasets/kukuroo3/body-performance-data Метка класса: class. Класс отрицательный – 1 (Классы A и B), класс положительный – (Классы C и D).

4. Оценка вероятности пожара по детекторам физических характеристик среды: https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset Метка класса: Fire_Alarm. Класс отрицательный – 0 (нет огня), класс положительный – 1 (пожар).

Вывод

В результате выполнения данной лабораторной работы мы познакомились с Google Colab и фреймворком Apache Spark. Были выполнены две программы, использующие различные компоненты фреймворка Apache Spark, что позволяет оценить эффективность и применимость этой технологии в решении задач обработки и анализа данных. Общий анализ показывает, что Apache Spark является мощным инструментом для работы с большими данными, способным обеспечить не только высокую скорость обработки данных, но и глубокие аналитические возможности в области машинного обучения. Это делает его пригодным для использования в широком спектре задач.

Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

Выполнение задания

Выполнено меньше половины задания – 0 баллов.

Большей частью выполнено – 2 балл.

Выполнено полностью – 4 балла.

Качество отчета

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету – 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований – 2 балл.

Соответствует всем требованиям – 4 балла.